



Module Code & Module Title
CS5002NT – Software Engineering

Assessment Weightage & Type
Individual Coursework (35%)

Year and Semester
2025-26 Autumn
30 - Credit

Student Name: Abhishek Goswami
London Met ID:24045789
College ID:np05cp4a20216
Assignment Submission Date: April-30-2026
Assignment Due Date: April-30-2026
Submitted To: Mr. Amit Shrestha

I confirm that I understand my coursework needs to be submitted online via my second Teacher under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

Table of Contents

1	Introduction	1
1.1	Introduction to Software Engineering	1
1.2	Introduction to Coursework	1
2	WORK BREAKDOWN STRUCTURE (WBS) and GANTT CHART	2
2.1	Work Breakdown Structure	2
2.2	Gantt Chart	3
3	Use Case Model.....	5
3.1	Use Case Diagram.....	5
3.2	High Level Use Case Descriptions.....	7
3.3	Expanded Use Case Descriptions	13
4	COLLABORATION DIAGRAM	16
4.1	Collaboration Diagrams	16
4.2	Collaboration Diagram: Book Table	17
5	Activity Diagrams	18
5.1	Overview of Activity Diagram	18
5.2	Activity Diagram: Place Order	18
6	CLASS DIAGRAM.....	20
6.1	Domain Class Identification.....	20
6.2	Analysis Class Diagram	21
7	FURTHER DEVELOPMENT PLAN.....	23
7.1	Architectural Choice.....	23
7.2	Design Pattern	24
7.3	Development Plan.....	25
7.3.1	Tools and Technology.....	25
7.3.2	Development Priority Order	26

7.4	Testing Plan	26
7.5	Maintenance Plan	27
8	Prototype.....	28
9	CONCLUSION	43
10	References	44

Table of Figure

Figure 1: Work Breakdown Structure (WBS) - Gokyo Bistro Management System....	3
Figure 2: Gantt chart	4
Figure 3: Use Case Diagram - Gokyo Bistro Management System	6
Figure 4: Collaboration Diagram – Book Table Use Case.....	17
Figure 5: Activity Diagram – Place Order Use Case.....	19
Figure 6: Analysis Class Diagram – Gokyo Bistro Management System	22
Figure 7: Prototype 1 Landing Page.....	28
Figure 8: Prototype 2 User registration page.....	29
Figure 9: Prototype 3 Login Page.....	30
Figure 10: Prototype 4 Member Dashboard	31
Figure 11: Prototype 5 Admin Dashboard	32
Figure 12: Prototype 6 Page of a Book Table.	33
Figure 13: Prototype 7 Payment Page	34
Figure 14: Prototype 14 Page of offers and discounts (Admin).....	41
Figure 15: Prototype 15 Bill / Receipt Page.....	42

Table of Table

Table 1: High Level Use Case Book Table.....	7
Table 2: High Level Use Case Login and Registration	8
Table 3: High Level Use Case Make payment	8
Table 4: High Level Use Case Set Beverage Choices	9
Table 5: High Level Use Case Cancel Reservation	9
Table 6: High Level Use Case Place Order.....	10
Table 7: High Level Use Case Update Menu	10
Table 8: High Level Use Case Generate Financial Report.....	11
Table 9: High Level Use Case Rate Experience	11
Table 10: High Level Use Case Publish Offer	12
Table 11: High Level Use Case Generate Bill	12
Table 12: Expanded Use Case Description - Book Table	13
Table 13: Detailed Use Case Description - Place Order	15
Table 14: Gokyo Bistro Use Cases - Domain Classes	20

1 Introduction

1.1 Introduction to Software Engineering

Software engineering is the application of engineering disciplines to the design, development, maintenance and operation of software systems (Sommerville, 2016). Software engineering differs from programming-in-the-small in that it incorporates formalized methods, quality management, and project management to produce robust and maintainable software systems. The discipline has grown considerably over the past few decades to accommodate the ever-growing complexity of today's software systems and the vital role of software in commercial, government, and social systems.

Software engineering addresses the entire software development life cycle (SDLC), including requirements elicitation, analysis, design, coding, testing and post-delivery maintenance. The field draws on object-oriented analysis and design (OOAD) as a major paradigm and uses the Unified Modelling Language (UML) to precisely and unambiguously describe structural and behavioral aspects of software systems. Good software engineering practice helps avoid project failure, boosts programmer productivity, and results in systems that are both technically and financially viable.

1.2 Introduction to Coursework

This assignment represents the individual piece of assessment for the CS5002NT Software Engineering module, which makes up 35% of the module grade. The task involves employing object-oriented analysis and design techniques to a commercial case study and is therefore a combination of theoretical software engineering principles and system development practices.

This assignment is based on Gokyo Bistro, a renowned restaurant in Jhamsikhel, Lalitpur. The establishment is an indoor-outdoor restaurant and has recently seen a dramatic increase in customer requests. This has revealed serious limitations in the manual operation system: frequent over-booking has resulted in customer turnback, take-out service is not being fulfilled, and there is no systematic record of table reservations. The restaurant thus needs a comprehensive IT solution to automate table booking, beverage ordering, payment, order taking, menu management, accounts and reporting, promotions, and customer support activities.

This report offers a set of OOAD artefacts for the proposed Gokyo Bistro Management System. This report is structured as follows: a Work Breakdown Structure (WBS) and Gantt chart to define the project scope and schedule; a Use Case model, with a high level description and an expanded description of use cases; a Collaboration diagram; an Activity diagram; an Analysis class diagram; a further development plan, including architecture, design patterns, tools, testing and maintenance; and a series of interface prototypes for key system features.

2 WORK BREAKDOWN STRUCTURE (WBS) and GANTT CHART

2.1 Work Breakdown Structure

A Work Breakdown Structure (WBS) is a decomposition project management technique in which the total work is decomposed into smaller work packages representing deliverables or activities (WorkBreakdownStructure.com, 2019). A well-developed WBS serves as the foundation for project planning, facilitating effort estimation, task assignment and progress tracking. Figure 1 illustrates the WBS of the Gokyo Bistro Management System project, which breaks down the project work into eight major work packages, each representing a development sprint in the selected Agile-Scrum development process.

The eight work packages aligned to the sprints are: (1) Initiation Stage, which involves requirements gathering, system analysis, UML and UI/UX modelling, and development environment setup; (2) Sprint 1 - Authentication System, covering login and registration for Admins, Members, Visitors, and other role types; (3) Sprint 2 - Table Booking System, involving table availability, booking process, beverage selection, and booking validation; (4) Sprint 3 - Payment and Cancellation System, implementing advance payment collection, payment gateway integration, and cancellation fee logic; (5) Sprint 4 - Order and Delivery System, including order placement, home delivery, and order tracking; (6) Sprint 5 - Menu Management System, allowing administrators to add, edit, and remove menu items; (7) Sprint 6 - Billing and Reporting System, providing bill generation, receipt printing, and AI-based revenue forecasts; (8) Sprint 7 - Feedback and Offers System, enabling experience rating, feedback collection, and promotional offers publishing; and (9) Final Release, involving system testing, deployment, and documentation.

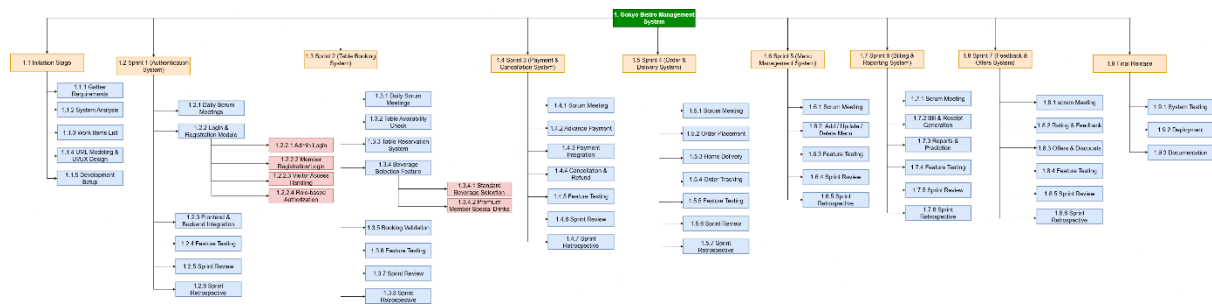


Figure 1: Work Breakdown Structure (WBS) - Gokyo Bistro Management System

2.2 Gantt Chart

The Gantt chart is a bar-chart scheduling tool that represents activities on a calendar, displaying the planned start and finish dates, duration and interdependencies of all project tasks in a single view (Kerzner, 2017). The Gantt chart in Figure 2 is based on the Agile-Scrum development approach. The sprint is a time-boxed iteration from Feb 1 to May 1, 2026. The time frame spans from Q3 2025 to Q2 2026, corresponding to the academic year project duration.

The Gantt chart illustrates typical Scrum ceremonies that are integrated into each sprint, such as daily Scrums, sprint reviews, and sprint retrospectives, along with the feature development tasks. The relationships between sprints are defined in terms of finish-to-start; for example, the authentication system sprint must be completed before the table booking sprint starts, as the booking system requires users to be logged in. The Final Release phase is planned following the completion of the feature development sprints, and includes system integration testing, production release, and final documentation.

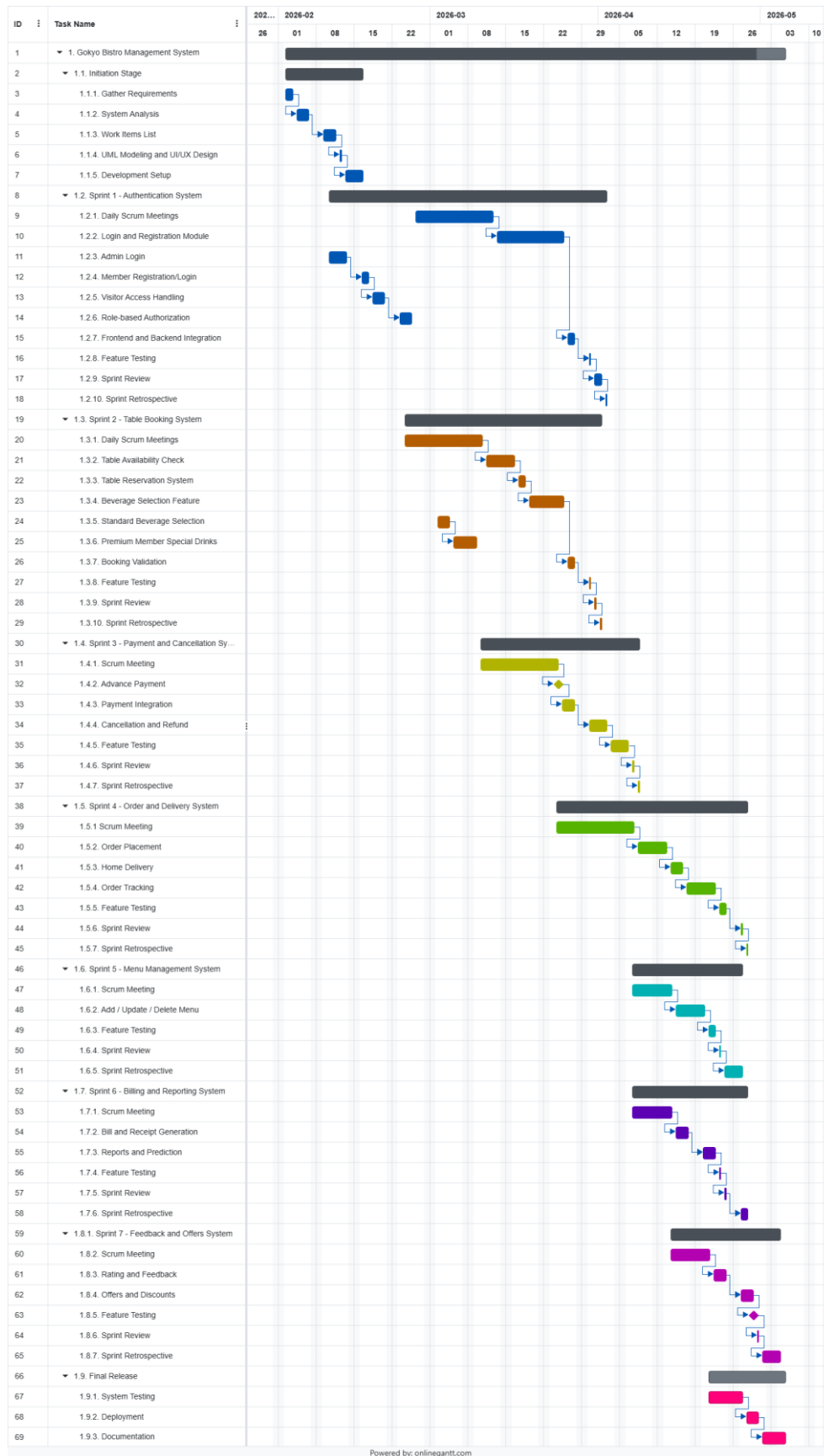


Figure 2: Gantt chart

3 Use Case Model

3.1 Use Case Diagram

A use case diagram is a behavioral UML diagram that represents a system's functional requirements by showing the actors that interact with the system and the use cases (unit functions of the system) those actors engage in (Cockburn, 2001) . The system boundary clearly defines what is in scope for the new system and what is out of scope. The main actors in the Gokyo Bistro Management System are: User (the general public who can access system features without being a member); Visitor (an unregistered user who can check table availability and view menu); and Member (a registered user with an active account who can access all features for booking, ordering and premium services). The admin actor oversees administrative activities of the system.

The use case diagram for the Gokyo Bistro Management System includes the following primary use cases (as shown in Figure 3): Book Table (extends to Check Table Availability and Set Beverage Choices); Make Payment (extends to Online Payment, Cash Payment, and Generate Receipt); Rate the Experience (extends to Give Feedback Message and decomposes into Rate Environment, Rate Food, and Rate Service sub-uses); Cancel Reservation (extends to Subject for Refund); Register Account (extends to Validate Details); Login to Account (extends to Check Credentials); Submit Order (extends to Check Item Availability, Pay Bill, and further extends to Deliver to Home and Dine-In); Analyze Revenue (extends to Generate Financial Report); and Publish Offer (extends to Add Special Offer and Add Discount). On the back-end, the admin actor is involved in the Manage Menu (with Add Item, Delete Item and Update Details sub-uses), Generate Bill, and Generate Financial Report use cases.

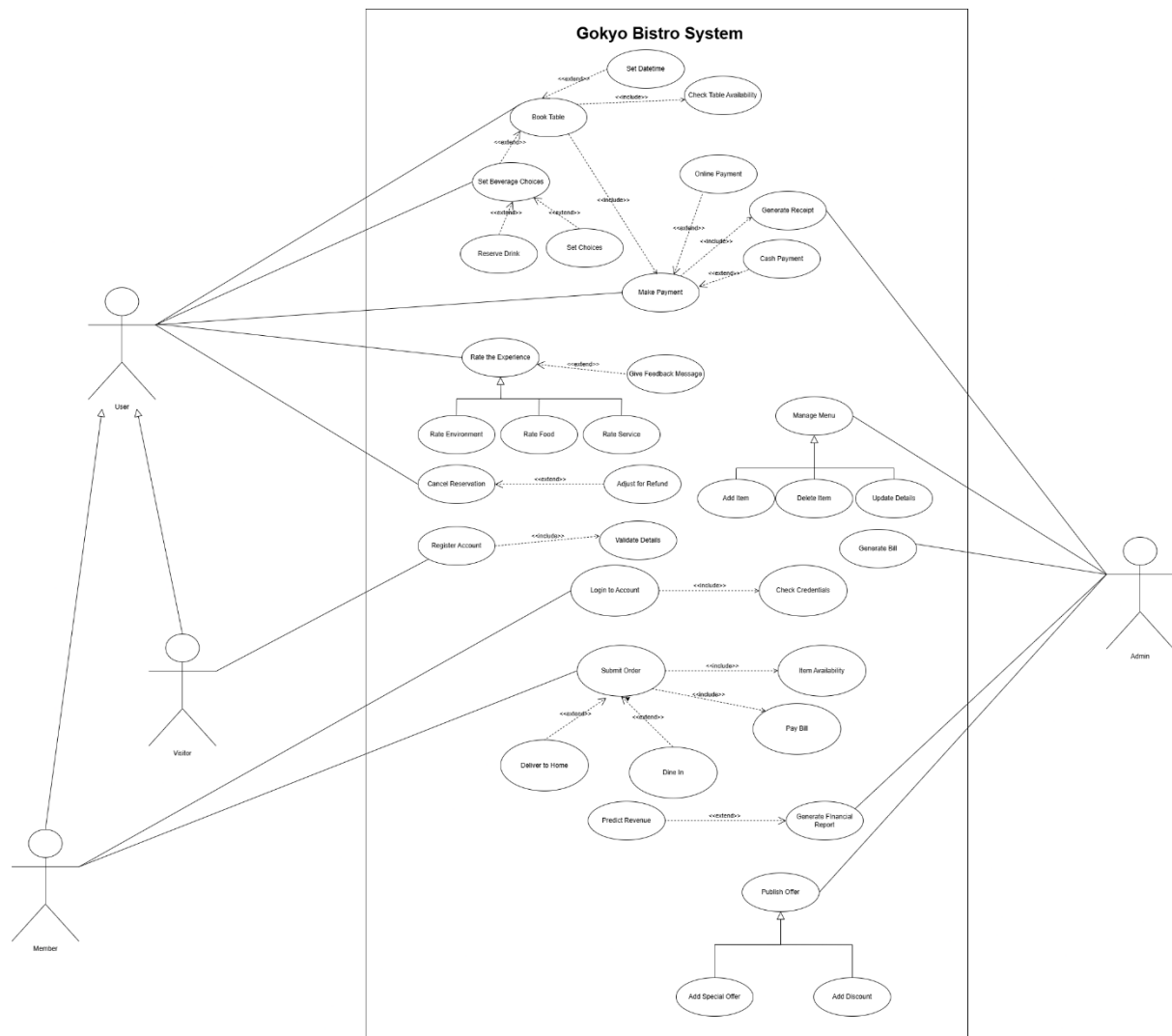


Figure 3: Use Case Diagram - Gokyo Bistro Management System

3.2 High Level Use Case Descriptions

High-level use case descriptions are succinct summaries of each use case, including the primary actor, a brief narrative description of the purpose and the critical interaction scenario. They are communication artefacts in the initial requirements elicitation process, allowing interested stakeholders to confirm the scope of proposed system features without yet having to define detailed interactions. The high-level descriptions of each major Gokyo Bistro use case are shown below.

Book Table

Table 1: High Level Use Case Book Table

Attribute	Detail
Use Case	Book Table
Actor	Visitor / Member
Description	The visitor or member goes to the book table page. Available dates and times are shown. The member chooses an available time and, if applicable, expected wine/whiskey or beer choices. Gold members can also reserve wine or whiskey choices. The reservation is stored and the user is charged an NPR 1,200 advance payment to finalise the reservation. The user receives a confirmation on successful booking.

Login and Registration

Table 2: High Level Use Case Login and Registration

Attribute	Detail
Use Case	Login and Registration
Actor	Visitor / Member
Description	Visitors can register for an account, supplying their name, phone number and a password. Registered Admin and Member users log into their accounts. The system checks the information provided which is created by a new user profile or finds an existing one and directs the authenticated user to their dashboard. Failure to login displays an error message.

Make Payment

Table 3: High Level Use Case Make payment

Attribute	Detail
Use Case	Make Payment
Actor	Visitor / Member
Description	After a booking or order confirmation, the user is presented with the payment screen. It displays the amount to pay and the available payment options (online or cash). In the case of online payments, the user enters card or e-wallet details, which are verified via a payment gateway. Payments are confirmed by generating a bill or receipt. The user is notified of the payment failure if the gateway rejects the payment.

Set Beverage Choices

Table 4: High Level Use Case Set Beverage Choices

Attribute	Detail
Use Case	Set Beverage Choices
Actor	Visitor / Member
Description	When booking a table, the user is given the option to select beverages in advance. Visitors and members can select from the list of beverage options. Premium members also have a select list of wines and whiskeys from which to make exclusive bookings. The system saves preferences to the booking.

Cancel Reservation

Table 5: High Level Use Case Cancel Reservation

Attribute	Detail
Use Case	Cancel Reservation
Actor	Visitor / Member
Description	A user with an active reservation can cancel the reservation. If the cancellation is made before the last day, the full NPR 1,200 advance is refunded. If the cancellation is raised on the day of the reservation, the system deducts the penalty (NPR 500) and refunds the remaining NPR 700. The system changes the status of the reservation and informs the user.

Place Order*Table 6: High Level Use Case Place Order*

Attribute	Detail
Use Case	Place Order
Actor	Member
Description	A member chooses items from the menu of food and beverage. The member enters the quantity for each item and chooses a delivery mode: home delivery or dine-in. If it's home delivery, the address is entered. The order is validated for available items, the order is summarised and payment is processed. Once payment is confirmed, the kitchen receives the order, and the member receives an email confirming the order.

Update Menu*Table 7: High Level Use Case Update Menu*

Attribute	Detail
Use Case	Update Menu
Actor	Admin
Description	The admin selects the menu maintenance page and can add new items by providing a name, category, price, and availability; update details of existing items; or delete outdated items. The system checks the data entered and commits the changes to the menu database, which are reflected in the menus displayed to customers in real-time.

Generate Financial Report*Table 8: High Level Use Case Generate Financial Report*

Attribute	Detail
Use Case	Generate Financial Report
Actor	Admin
Description	The admin logs into the reporting system and chooses a reporting period. The system collates data from the transaction records and calculates total income, sales by menu item, and total income for the same period last year. A sophisticated revenue forecasting engine uses past data to forecast income for future periods and suggests items for promotion. The report can be viewed on the screen or printed in PDF form.

Rate the Experience*Table 9: High Level Use Case Rate Experience*

Attribute	Detail
Use Case	Rate the Experience
Actor	Visitor / Member
Description	After an eat out or delivery service experience, the user can go to the feedback page and provide distinct ratings for Food, Service and Ambience on a discrete scale. Users may include an optional text message. The feedback is saved and rating data is made available to the admin for business analysis.

Publish Offer*Table 10: High Level Use Case Publish Offer*

Attribute	Detail
Use Case	Publish Offer
Actor	Admin
Description	The admin publishes offers by entering an offer name, description, discount percentage or amount, start date, end date and target audiences. Event-specific offers can also be set up. Upon publishing the offer, it is announced via the customer interface, with eligible customers being notified of promotions.

Generate Bill*Table 11: High Level Use Case Generate Bill*

Attribute	Detail
Use Case	Generate Bill
Actor	Admin / Member
Description	At the end of a dine-in session or after the confirmation of an order, the system generates the total bill amount, considering taxes, discounts, or promotional offers. The admin can print bills for dine-in transactions, and the system prints e-receipts for online orders. The bill includes a line item and total amount due.

3.3 Expanded Use Case Descriptions

Expanded use case descriptions provide a detailed description of the interactions between a user and the system for a particular use case. They specify the preconditions and postconditions, the primary success scenario and significant alternatives and exceptions, and are thus used as precise requirements specifications and test case sources (Cockburn, 2001). Below are two expanded case descriptions: Book Table and Place Order.

Expanded Use Case 1: Book Table

Table 12: Expanded Use Case Description - Book Table

Expanded Use Case Description: Book Table	
Actor	Visitor / Member
Description	The visitor or member logs into the table booking system to view availability, set beverage options and secure an appointment for a specific date and time. A deposit of NPR 1,200 is required to book a reservation.
Pre-condition	The visitor is a registered Member or unregistered Visitor. The system is up and running and the table availability information is up to date.
Post-condition	The user's booking is recorded as a confirmed reservation and advance payment is recorded. A confirmation email is sent to the user.
User Action	System Response

The user goes to the Book Table section.	The system presents the dates and times available.
The user chooses a date, time and number of guests.	The system confirms if the time slot is available or not.
If possible, the user can choose drinks from the list. Premium users are offered a wine/whiskey list.	The user's beverage preferences are logged against the session.
The user then proceeds to checkout and pays NPR 1,200 using an approved payment method. Payment is processed via the payment gateway.	If successful, the reservation is processed and a booking reference created.
The user is presented with booking confirmation and reference number.	An order confirmation is sent to the user's contact.

Expanded Use Case 2: Place Order*Table 13: Detailed Use Case Description - Place Order*

Expanded Use Case Description: Place Order	
Actor	Member
Description	The Member chooses items from the menu for food and beverages, selects delivery options (dine in or take home) and makes a payment. The kitchen receives the order, and an order confirmation is sent.
Pre-condition	The user is logged in as a registered Member. At least one menu item is available.
Post-condition	A confirmed order record is created. The system notifies the kitchen, generates the bill, and adjust inventory levels.
User Action	System Response
The member logs into the system and clicks on the Order menu.	The system retrieves and presents menu items including availability information.
The member adds items and quantities.	It checks if each item is available and includes it in the order summary.
The member selects a delivery option: Home Delivery or Dine-In.	If Home Delivery, member enters the address. For Dine-In, address is not required.

The member checks the order details and places the order.	The system shows the total amount to be paid including the charges.
The member chooses a payment option (online/cash) and pays.	The system processes the payment. It issues a bill/receipt and confirmation.
The member receives an order confirmation.	The kitchen is notified, and the order status updates to “confirmed”

4 COLLABORATION DIAGRAM

4.1 Collaboration Diagrams

A collaboration diagram (also known as a communication diagram in UML 2.x) is a structural-behavioral UML diagram that shows the interactions among objects that are needed to complete a given use case or operation. Unlike a sequence diagram, which lays out messages on a vertical time axis, the collaboration diagram lays out the objects in a free-form spatial format and uses numbered messages on links to show the sequence of interactions. This representation is well suited to display the object structure of the system, since it displays both the communicating objects and the sequence of communication.

The collaboration diagram has four types of elements. Objects, shown as rectangles with names, are instances of classes in the system. The Actor, depicted as a stick figure, initiates the interaction chain. Links, represented as lines between objects, represent a navigable association that allows the exchange of messages. Messages, indicated on links with hierarchical sequence numbers, specify the content and order of messages exchanged.

4.2 Collaboration Diagram: Book Table

The elaborated use case for Book Table was used as the source for the collaboration diagram. The objects in this interaction represent the domain classes identified in the analysis: the BookTableUI boundary object is the user interface; the BookTable control object is responsible for coordinating the actions required to make a booking; and the five entity objects (browseDetails, Availability, beverage, payment and reservation) represent the persistent data objects of this system.

The interaction, depicted in Figure 4, takes place as follows. The User actor sends message 1 (requestBooking) and message 2 (selectBooking) to the BookTableUI boundary object. BookTableUI sends message 1.1 (getBookingDetails), 2.1 (verifySlots), 3.1 (verifyBeverageList) and 4.1 (verifyPaymentDetails) to the BookTable control object. The BookTable control replies to the BookTableUI with display messages 1.1.2, 2.1.2, 3.1.2 and 4.1.3. Meanwhile, the BookTable control sends delegation messages to the entity objects: 1.1.1 fetchBookingDetails to Booking, 2.1.1 getAvailabilityStatus to Availability, 3.1.1 beverageDetails to beverage, 4.1.1 payment to payment, and 4.1.2 booking to reservation. This number identifies the delegated nature of the booking interaction, and how user interaction results in interaction between multiple objects.

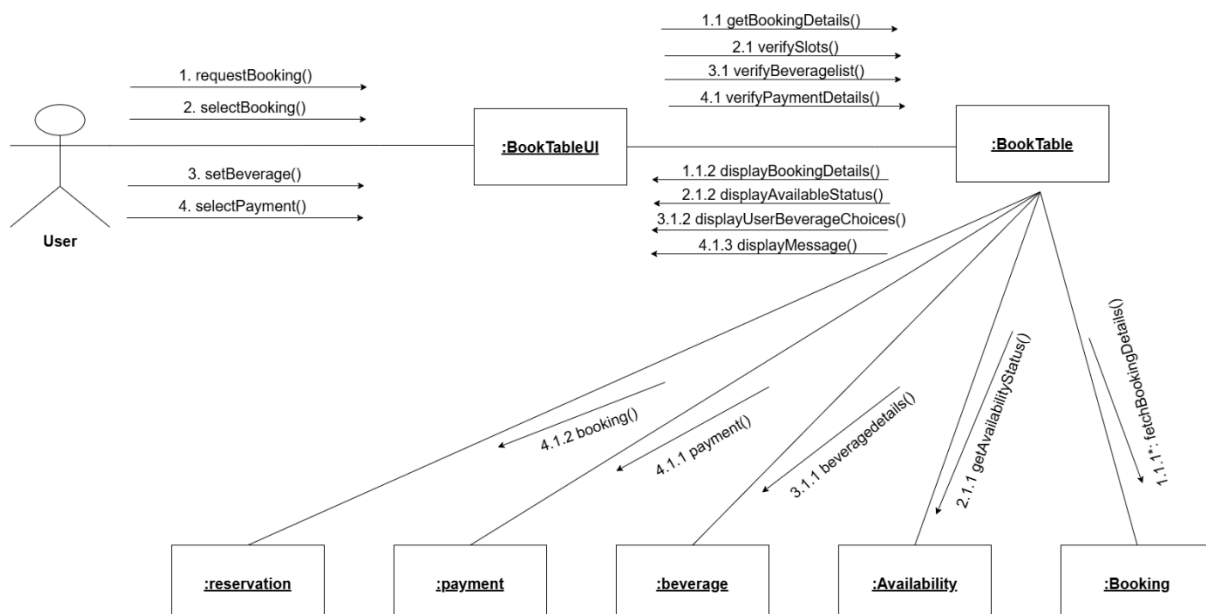


Figure 4: Collaboration Diagram – Book Table Use Case

5 Activity Diagrams

5.1 Overview of Activity Diagram

An activity diagram is a UML behavioral diagram that represents the dynamic flow of control and data in a system or process. It is an enhanced form of a flowchart, offering the added ability to model parallelism with fork and join bars, swimlanes for delineating responsibilities and object flow for object transformations (OMG, 2017). Activity diagrams are commonly used to model business processes and use case scenarios, as well as to represent algorithms, and are a good accompaniment to the static structural view offered by class and object diagrams.

The notational elements include solid circles for the initial and final nodes; rounded rectangles for the activity states; diamonds for the decision and merge nodes; bars for the fork (parallelism start) and join (parallelism end); and vertical swimlane divisions for the delineation of activities to actors or system components.

5.2 Activity Diagram: Place Order

The activity diagram in Figure represents the Place Order use case (not depicted in the collaboration diagram). The diagram uses two swimlanes - Visitor/Member on the left and System on the right - to separate user actions from system responses.

The process starts with the user logging in to the system. The menu items are shown in the System swimlane. A decision node tests whether the items selected are available: if not, the system displays a message and the user selects other items; if yes, the system lists the items. The user then decides on the items and quantities to be ordered, as well as the delivery method. A second decision node evaluates Home Delivery: if chosen, the system captures the delivery address; if not, the system sets up the order to be eaten in. Both branches converge before an order summary is present. The user approves the order, followed by another decision point on the payment option: Internet payment triggers the payment gateway, with a subsequent decision on whether payment is successful or not; cash payment skips this step. In the case of a successful payment, a fork node triggers two activities - Generate Bill and Notify Kitchen. Both activities join at a join node, and the system sends an order confirmation. The user receives confirmation of the order and the process is completed.

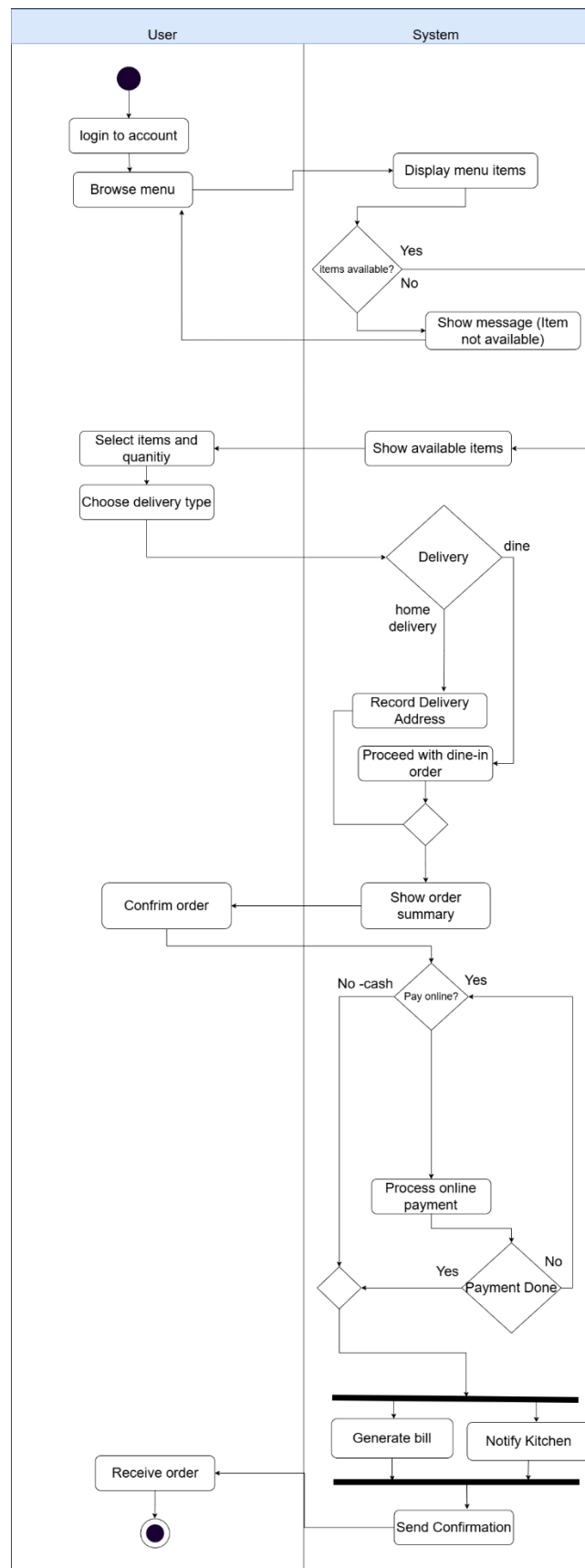


Figure 5: Activity Diagram – Place Order Use Case

6 CLASS DIAGRAM

6.1 Domain Class Identification

A UML analysis class diagram models the domain of a system, specifying the classes, attributes, operations and relationships between them. It is based on domain modelling, where candidate classes are identified by analysing the nouns and noun phrases in all the use cases and their descriptions. The class diagram is a key component in an object-oriented design, as it provides the template from which implementation classes are developed.

The domain classes extracted from the Gokyo Bistro use cases are listed in Table below.

Table 14: Gokyo Bistro Use Cases - Domain Classes

Use Case	Domain Class(es)
Login & RegistrationUser	Visitor, Member, Admin
Book Table	Reservation, Table, BeverageChoice
Set Beverage Choices	BeverageChoice, Menu, MenuItem
Make Payment / Cancel Reservation	Payment, Bill, Delivery
Place Order	Order, OrderItem, Delivery
Update Menu	Menu, MenuItem
Generate Financial Report	FinancialReport, PredictRevenue
Generate Bill	Bill, Order, OrderItem
Rate the Experience	Rating
Publish Offer	Offer

6.2 Analysis Class Diagram

The analysis class diagram in Figure 6 includes 18 domain classes with their respective attributes and operations, along with a rich array of relationships. The diagram captures the structural needs of the system and fulfils the minimum requirement of two inheritance, two aggregation and two composition relationships, in addition to several associations with multiplicities.

Inheritance is exhibited through: Visitor and Member classes inherit from the abstract class User, both sharing the identity attributes (userID, username, password) and allowing role-specific behaviours. Member is further specialised as PremiumMember, which introduces membershipLevel and a wineSelection() operation, capturing the enhanced privileges of the premium level membership.

Composition is used between Order and OrderItem (an OrderItem cannot exist without its parent Order), and between Bill and its sub-constituents (a Bill is inseparable from the transaction it captures and doesn't exist in independent of this). These are examples of the tight lifecycle association that composition entails.

Aggregation is employed between Menu and MenuItem (a MenuItem is a component of a Menu but can be conceptually regarded as an item catalogued independently of any menu instance), and between FinancialReport and PredictRevenue (the prediction sub-module is part of the reporting module but has its own lifecycle during reporting analysis).

Other links include: Reservation is associated with Table (a Reservation is for one table, a table can be booked by many reservations); Reservation references BeverageChoice (optional, pre-selected beverage choices); Payment is associated with Reservation and with Order; Delivery is associated with Order; Rating is associated with User; and Offer is associated with Admin (offers are created by an Admin) and MenuItem (offers apply to menu items). All association ends are labelled with multiplicities, e.g. 1..* (one to many), 0..1 (zero to one, denoting optional relationships), and * (many).

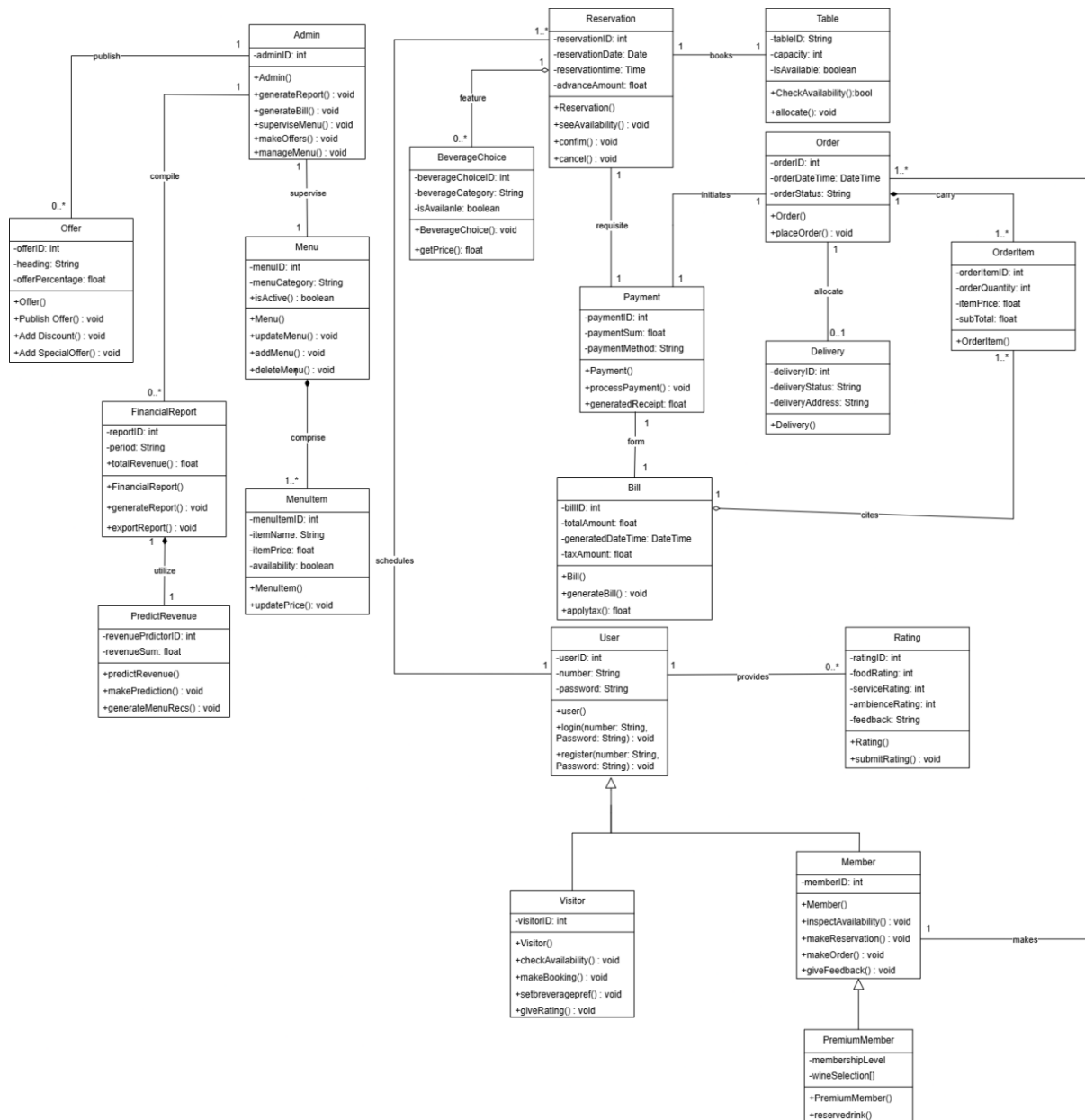


Figure 6: Analysis Class Diagram – Gokyo Bistro Management System

7 FURTHER DEVELOPMENT PLAN

The analysis and design artefacts - the WBS, Gantt Chart, Use Case Model, Collaboration Diagram, Activity Diagram and Class Diagram - are now in place and the project is ready to move to design and construction. The next sections outline the plan for the transition from analysis artefacts to the design, development, testing and maintenance of the system using the Agile-Scrum development process.

The Agile-Scrum development methodology was chosen for this project as its incremental, iterative approach fits Gokyo Bistro's requirements (Sutherland, 2020). The system's requirements are complex but well defined at the feature level, allowing features to be delivered in sprints. Scrum's lean meetings (sprint planning, daily stand-ups, sprint reviews and retrospectives) offer regular feedback and stakeholder visibility. Features can be prioritised and delivered in a staggered manner to make the restaurant operational with early features (such as table booking) while others are added in subsequent sprints.

7.1 Architectural Choice

The restaurant system will be built with a three-tier layered architecture, including Presentation Tier, Business Logic Tier and a Data Tier. We chose this architecture pattern because it ensures a separation of concerns: the presentation tier is responsible for the rendering of the user interface and capturing of user input; the business logic tier is responsible for encapsulating all the application rules, validations and workflow; and the data tier is responsible for persisting and retrieving data using an Object-Relational Mapping (ORM) layer. This allows for each tier to be modified - for instance, the user interface - without requiring changes to the other tiers, thereby enhancing maintainability and testability.

In this three-tier architecture, the Presentation Tier will be realised as a responsive web client via React.js, with a single source code running on both desktop and mobile web browsers. The Business Logic Tier will be implemented as a RESTful API, using Node.js and the Express.js framework, which will provide consistent HTTP endpoints for the front-end to access. The Data Tier will leverage a PostgreSQL relational database, using the Sequelize ORM, which enables transactional support for storing payment and reservation records. This system also supports horizontal scaling of the API tier, should demand increase, to meet the restaurant's growth projections.

7.2 Design Pattern

The major design pattern chosen for this project is Model-View-Controller (MVC) (Erich Gamma, 1994). MVC separates the application into three interconnected parts: the Model that represents data and business logic; the View that displays data to the user and receives input; and the Controller that responds to user actions (events) by performing operations on the model and preparing the appropriate views for display .

For the Gokyo Bistro system, each domain entity (Reservation, Order, Payment, Menu, User) will have a Model class representing its attributes and data operations. The View is composed of front-end React components. Express.js routes make up the Controllers, handling HTTP requests and interacting with models. MVC was chosen as it supports concurrent development by multiple developers working on different views and business logic; it enhances testing as both controllers and models can be tested independently from one another; and it yields a modularised code base where changes made to the user interface do not affect the business logic.

Furthermore, the Repository Pattern will be used in the data tier to enable a separation of database access logic from the services. This allows the business logic tier to interact with domain objects rather than SQL statements; thus, maintaining the separation of concerns and allowing for future database changes or schema migrations.

7.3 Development Plan

7.3.1 Tools and Technology

The development process will use the following tools and technologies:

- Front-End: React.js (Tailwind CSS to style it), Axios to communicate with an HTTP client, React Router to navigate a single-page application.
- Back-End: Node.js environment and Express.js web framework for RESTful API. JSON Web Tokens (JWT) will be used for stateless authentication and authorisation.
- Database: Primary relational database will be PostgreSQL. Sequelize ORM will be used to define schemas, migrations, and queries. Redis will be used for caching and limiting access.
- Payment Gateway: SDKs for an online payment gateway (such as eSewa or Khalti, widely used in Nepal) will be used to accept advance payments.
- Version Control: Git + GitHub to manage source code, review via pull request, and test with GitHub Actions.
- Project Management: Jira to plan sprints, manage the backlog, and track issues, where the WBS breakdown is mapped to Jira epics and stories.
- Testing: Jest and Super test for unit and integration testing of the back-end; React Testing Library for front-end testing; Selenium WebDriver for automated end-to-end tests.
- Deployment: Docker containers deployed on cloud hosting (e.g. AWS EC2, Digital Ocean Droplet) with Nginx reverse proxy.

7.3.2 Development Priority Order

The sprint development process is based on the priority order in the WBS and Gantt Chart. Sprint 1 (Authentication System) is the most important sprint as it enables the other system features. Sprint 2 (Table Booking System) is the second-highest priority because table booking is the most urgent problem for the restaurant. The next one is Sprint 3 (Payment and Cancellation) which is immediate because the payment is a requirement to confirmed reservations. Sprint 4 (Order and Delivery System) and Sprint 5 (Menu Management System) fulfil the home delivery need. Sprint 6 (Billing and Reporting) provides reporting to the administration. Sprint 7 (Feedback and Offers) is the last layer to be completed in customer engagement. The Final Release sprint covers end-to-end system testing, security upgrades, deployment and documentation.

7.4 Testing Plan

There will be a comprehensive testing plan that will involve four levels of testing to verify the quality, reliability and correctness of the system .

Unit Testing

All the individuals and methods will be tested in Jest through unit testing. In a test-driven development (TDD) approach, unit tests will be written in parallel with feature code, with the behaviour of each function in isolation being verified. External dependencies (like the database and payment gateway) will be replaced with mock objects when executing the unit tests so that the test speed and determinism can be preserved.

Integration Testing

Integration tests will ensure that modules assembled are interacting correctly. REST API endpoints will be tested with Supertest and checked against an in-memory or test-schema database instance and ensure that request response cycles properly cross the controller, service and repository layers. The booking–payment and order–delivery integration paths will be considered particularly because they imply multi-step chains of transactions.

System Testing

Selenium WebDriver will be used to script end-to-end system tests that will be run in a browser environment to simulate the entire user workflows. The main use cases (user registration and login, booking a table with the payment option, making an order with a home delivery option, generation of an administrative report, and publishing an offer) will be covered with test scenarios. The staging deployment setup will be simulated on the system tests against the production configuration.

User Acceptance Testing (UAT)

Before the Final Release sprint, Gokyo Bistro representative staff will be involved in a formal UAT exercise. Scripted scenario walkthroughs will be performed by testers and defects; usability problems and feature gaps will be documented. All defects that are critical and high priority that are detected during UAT will be eliminated prior to production deployment.

7.5 Maintenance Plan

Maintenance, including post-deployment, will be under a coordinated scheme that includes corrective, adaptive, and preventive effort .

Constant supervision of application logs and error tracking will be considered to be the support of corrective maintenance that can be ensured through the use of a tool like Sentry that will alert in real-time on the production exceptions. Reported defects will be noted in a special maintenance backlog and sorted by severity. Significant bugs will cause emergency hotfix releases within 24 hours; minor ones will be dealt with during the next planned maintenance.

Adaptive maintenance will be planned as quarterly releases to suit the changing business needs - such as adding more payment gateways, a new menu type or an extension of the premium membership plan. All adaptive releases will go through a similar sprint development and testing process as the original build.

Monthly database index optimisation and query performance analysis will be in the category of preventive maintenance, quarterly security audit such as dependency vulnerability scanning (npm audit) and authentication endpoint penetration testing, and annual assessment of the server infrastructure to ensure growth in user demand does

not surpass its scaling capacity. Backups of databases will be automated every day with a 30-day retention period. Recovery time objectives (RTO) and recovery point objectives (RPO) will be established in a disaster recovery plan to protect against data loss.

8 Prototype

Software prototypes refer to low-level, physical models of the suggested system interfaces and serve to have stakeholders validate requirements, identify design errors, and refine the user experience in an iterative way before full implementation (IBM, 2020). The Gokyo Bistro Management System interface prototypes have been developed as follows and take care of all the key features as outlined in the detailed specification. Every prototype has a short explanation of its purpose and main interface components.

Prototype 1: Landing Page

All visitors come in on the landing page. It features the Gokyo Bistro brand name, hero banner with high-quality interior photo, and call-to-action buttons, which are large and clearly displayed: Book a Table and Order Now. The menu, contact information, and login/register are accessible via navigation links. A top page promotional banner reveals any promotional deals made by the Admin.

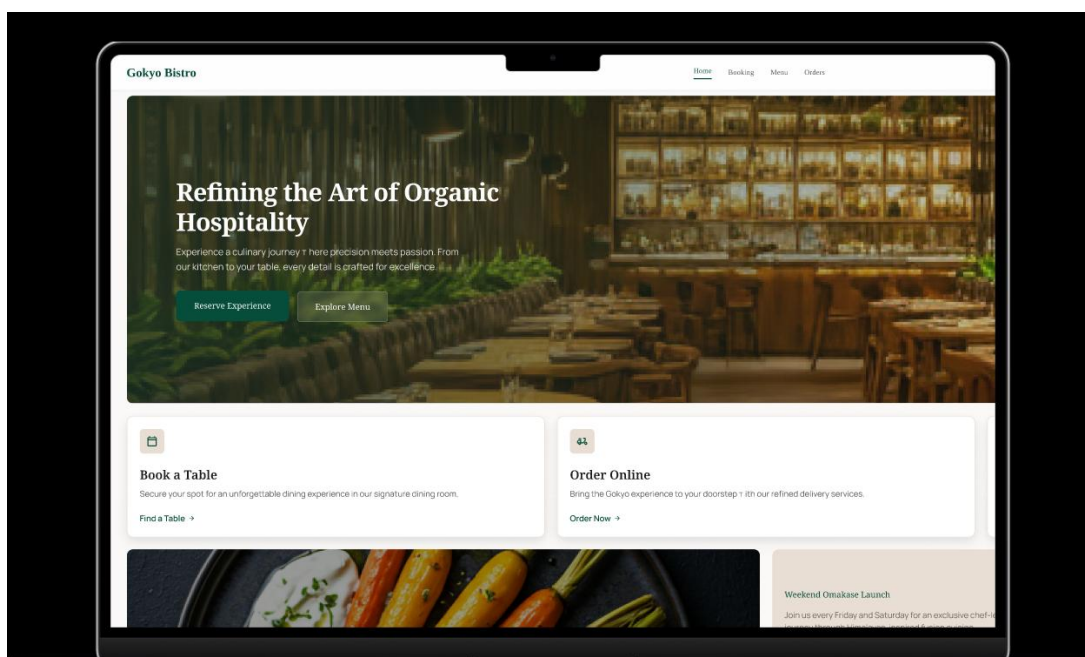


Figure 7: Prototype 1 Landing Page

Prototype 2: User registration page

The registration page has a clean form that requires the full name of the user, email address, contact number and a password that has a confirmation field. Input validation indicators give instant responses on whether fields are completed. At the end of the form is a Register button and a link to the bottom of the page sends the form to the login page. It has a terms-and-conditions acknowledgement checkbox to obtain consent.

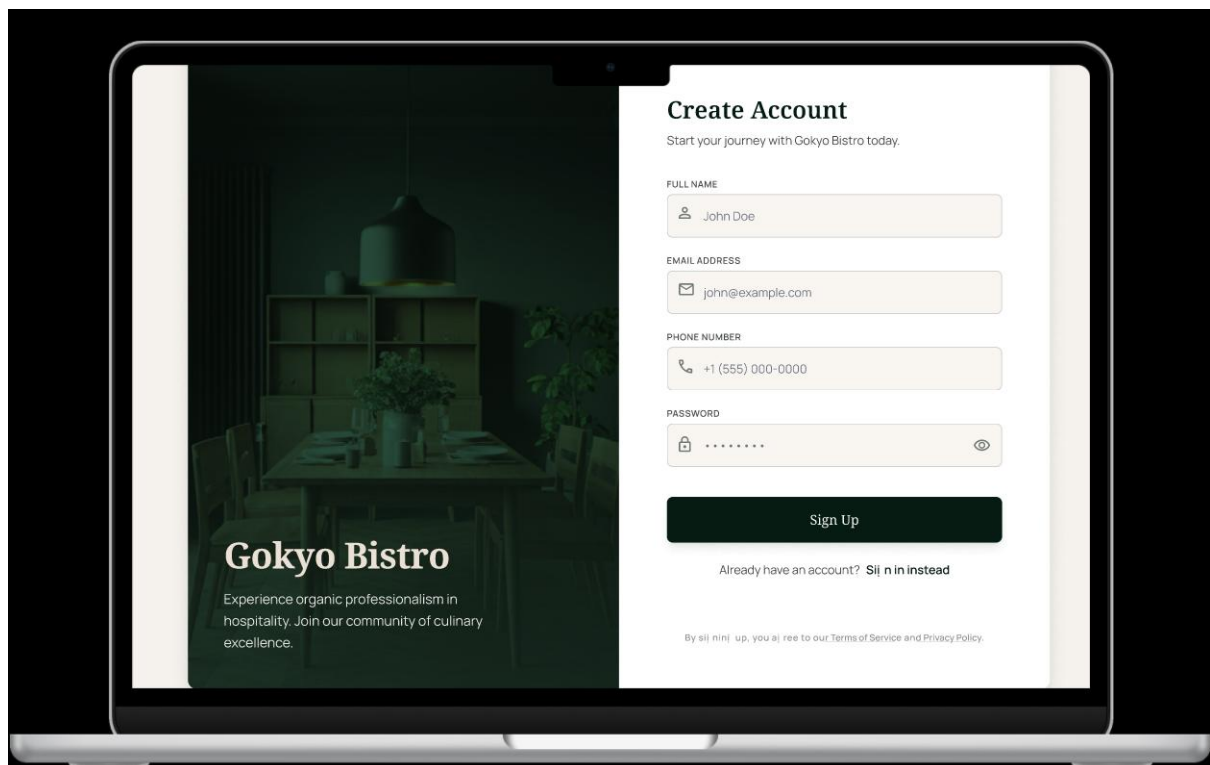


Figure 8: Prototype 2 User registration page

Prototype 3: Login Page

The login page has email and password boxes and a login button. Admin, Member and Visitor have distinct paths to log in clearly marked. A Forgot Password button triggers a password-reset process. When wrong credentials are entered, an error message will be displayed on the screen, but without page refresh.

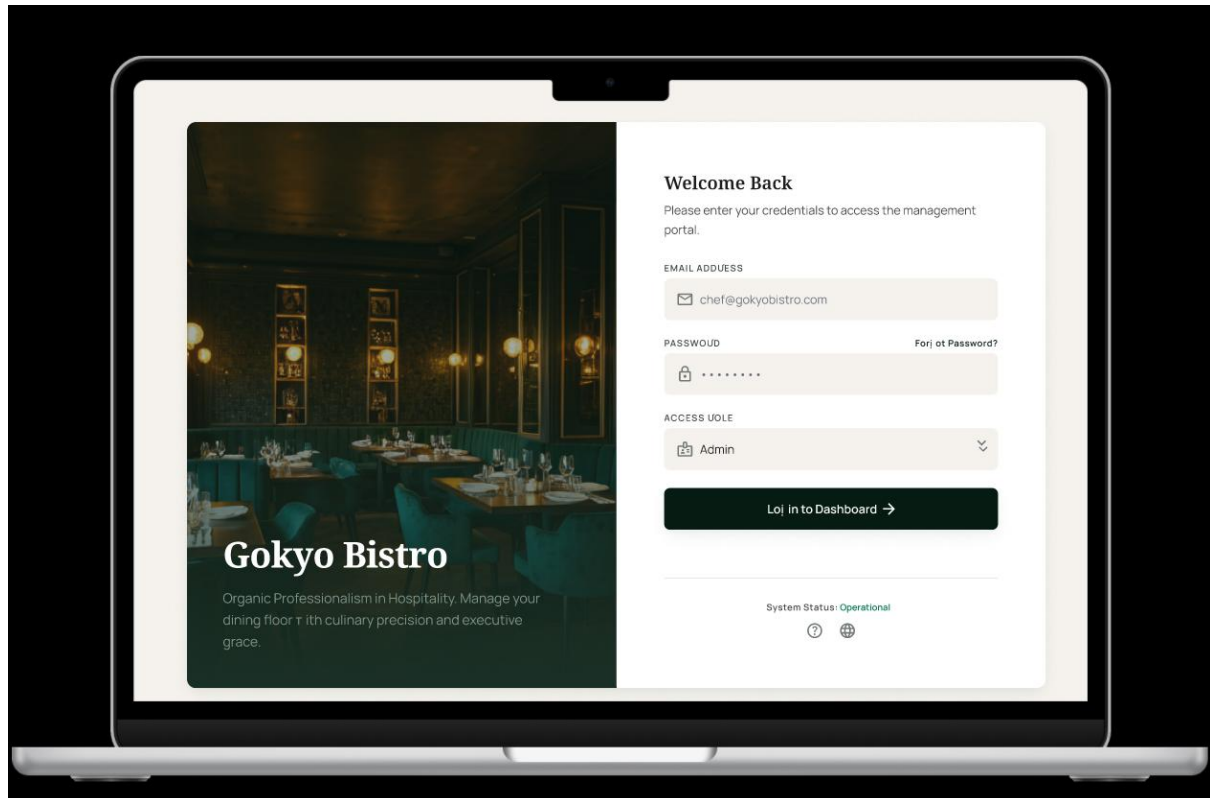


Figure 9: Prototype 3 Login Page

Prototype 4: Member Dashboard

The member dashboard will have a customised welcome head showing the name and membership type (standard or premium) of the member. Quick-access cards guide the user on the active reservations, recent orders, loyalty points (premium), and unread notifications. There is a sidebar menu where all the member functions can be found: Book Table, Order Food, My Reservations, Order History, My Ratings, and Account Settings.

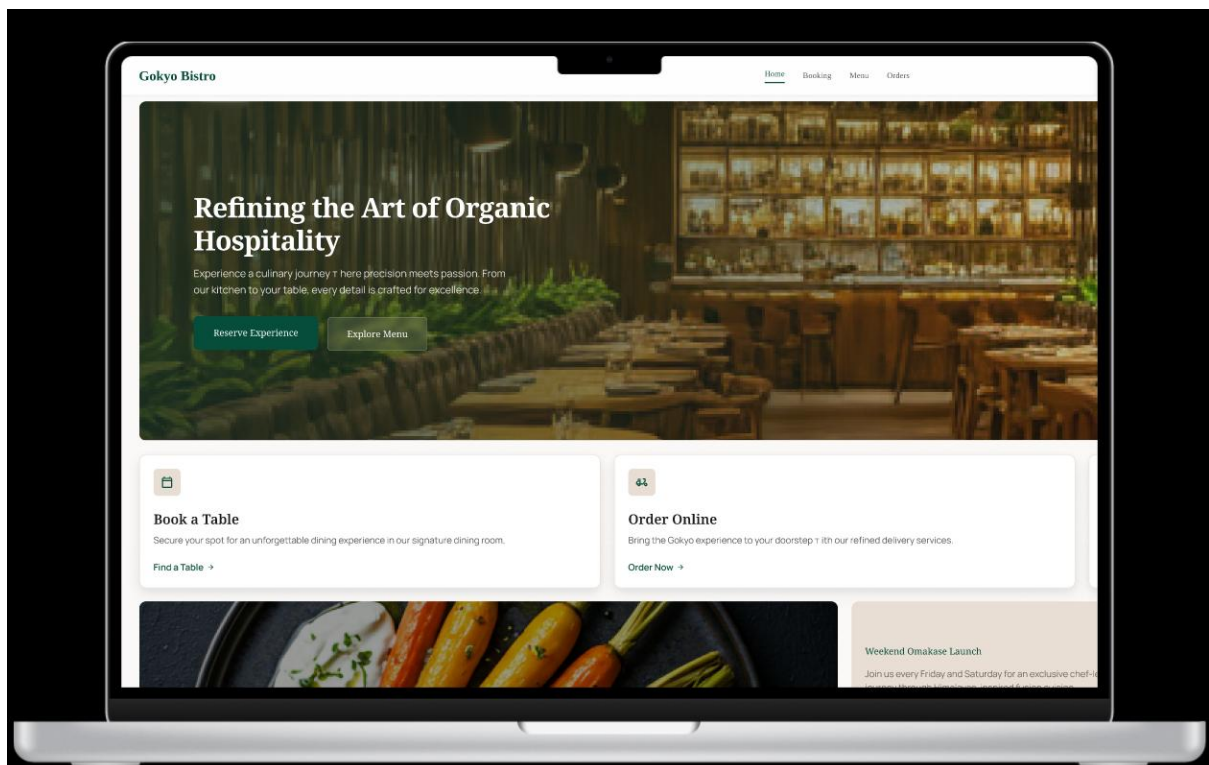


Figure 10: Prototype 4 Member Dashboard

Prototype 5: Admin Dashboard

The admin dashboard provides management overview containing summary statistics: the number of bookings made today, orders, the revenue collected this week, and requests to cancel the orders. Graphical widgets show the booking trends and revenue by category. All the administrative modules are accessible via navigation links: Menu Management, Reservation Management, Order Management, Reports, Customer Management, and Offers.

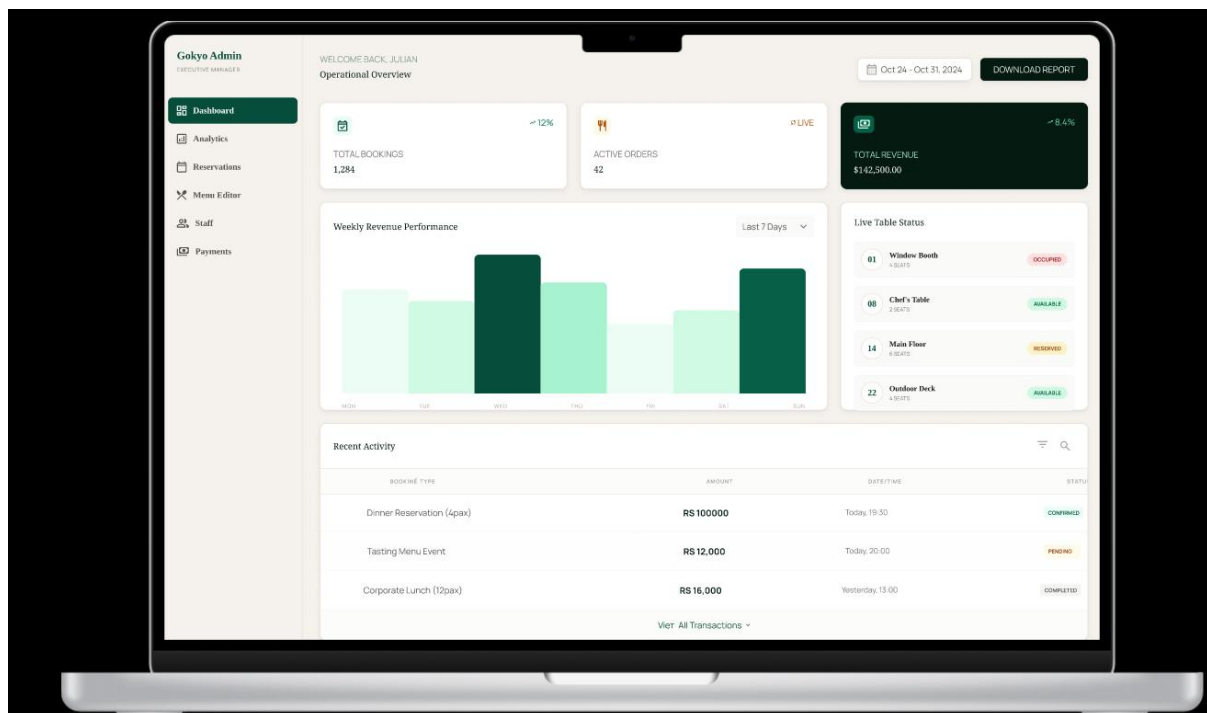


Figure 11: Prototype 5 Admin Dashboard

Prototype 6: Page of a Book Table.

The table booking page will offer a date-picker and time-slot multi-picker, where the status of each slot will be shown (available, limited, or fully booked). Party size is set by a numeric stepper. Below the slot option, there is an optional pre-selection beverage option with a list of beverages with checkboxes; premium members are offered an extra wine/whiskey reservation option. Selections are summarized in a booking summary panel to the right and the NPR 1,200 advance payment requirement is displayed in front of a Proceed to Payment button.

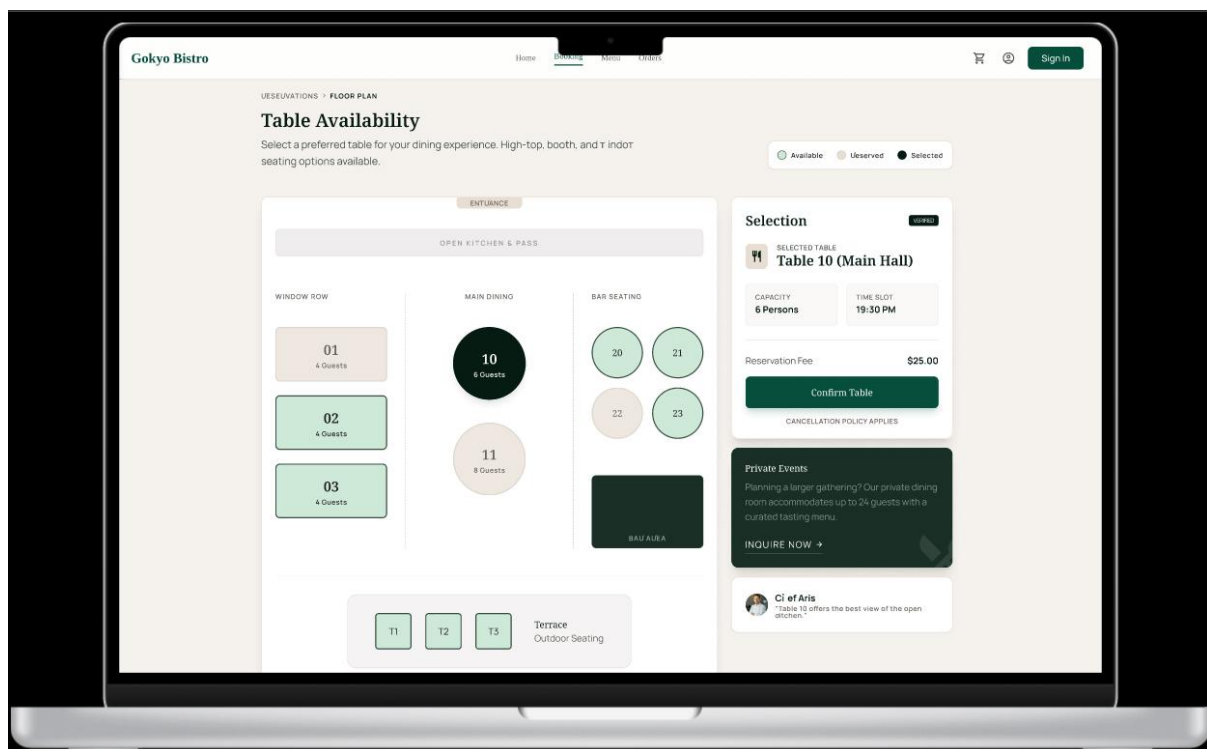


Figure 12: Prototype 6 Page of a Book Table.

Prototype 7: Payment Page

On the payment page, a summary of the transaction (booking or order reference, amount due) is shown. Payment method tabs will enable you to choose between eSewa, Khalti digital wallet and debit/credit card. The fields of card payment (card number, expiry, CVV) are displayed conditionally when choosing the method. The interface is complete with a secure payment mark as well as a pay NPR 1,200 confirmation button. On gateway rejection, a banner showing a payment failure with instructions to retry is displayed.

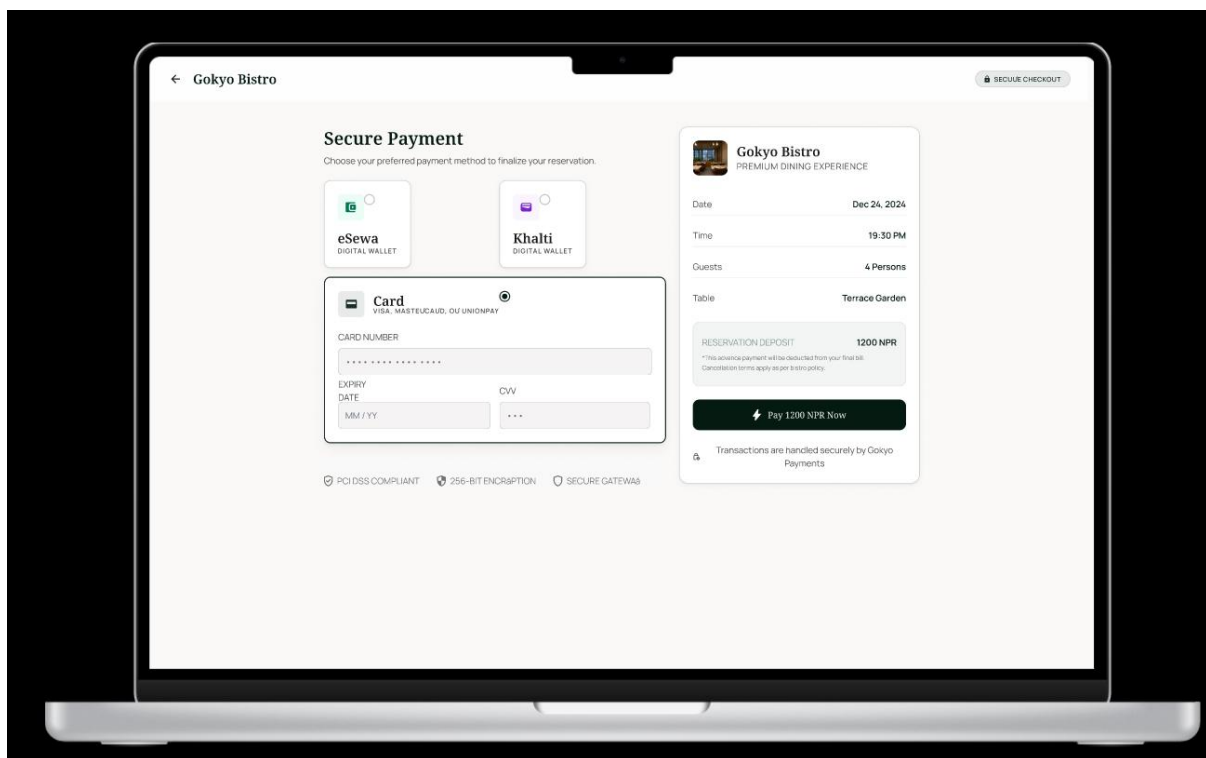
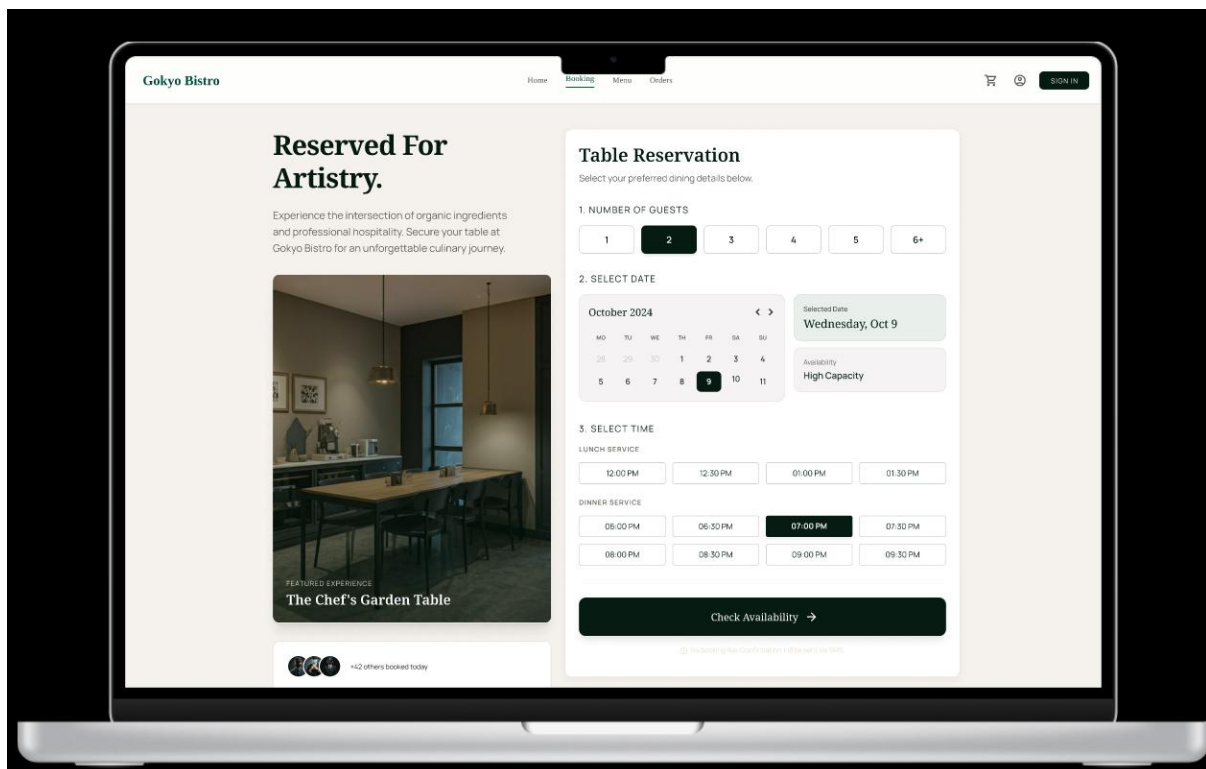


Figure 13: Prototype 7 Payment Page

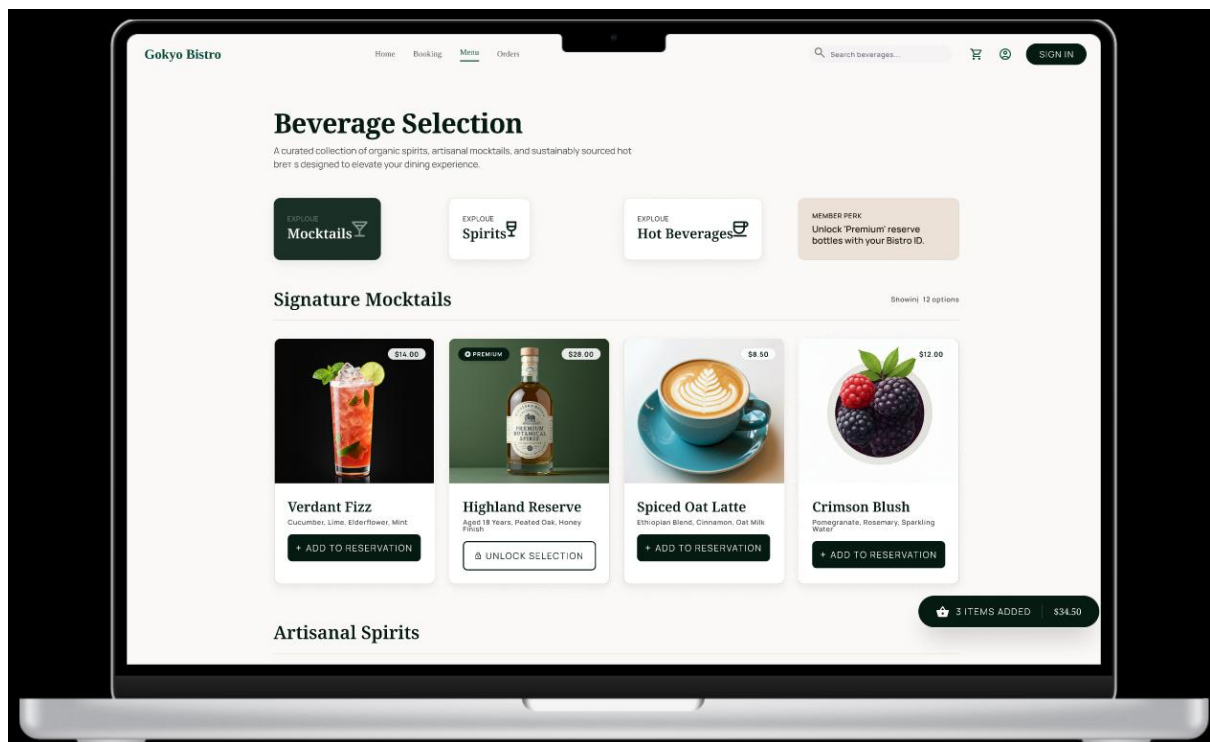
Prototype 8: My Reservations Page.

The reservations management page shows a tabular list of the active, future and past reservations of the user with booking reference, date, time, party size and status visible. A column of Actions offers the possibility of Cancel buttons of future reservations. The user confirms by a cancellation confirmation modal, which outlines the policy of the refund (full refund before the final day; NPR 500 deduction on the day).



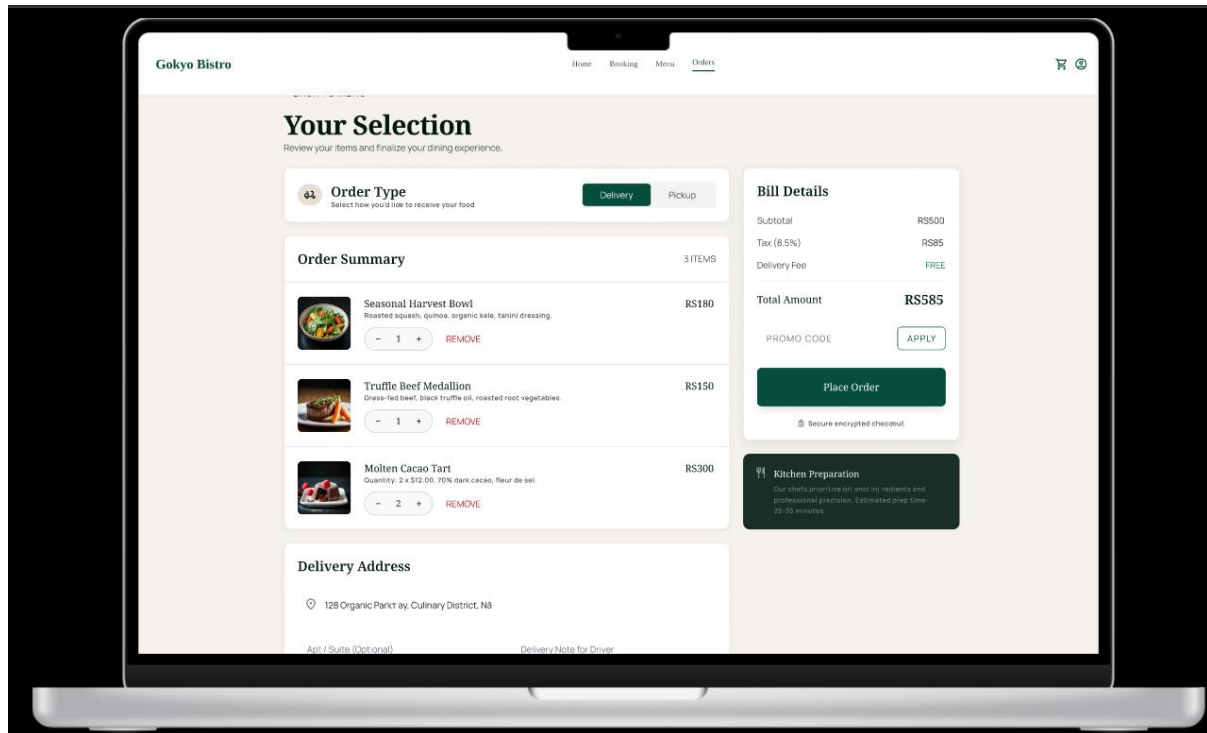
Prototype 9: Order Page / Menu.

The order page has menu items in category tabs (Starters, Mains, Beverages, Desserts). All item cards will contain an image, name, description, price and availability badge. Quantity steppers enable the selection of items in an incremental manner and the cumulative amount of the order shows in a sticky cart section at the right. There is a radio button (Home Delivery or Dine-In) that has been placed above the cart summary eminently.



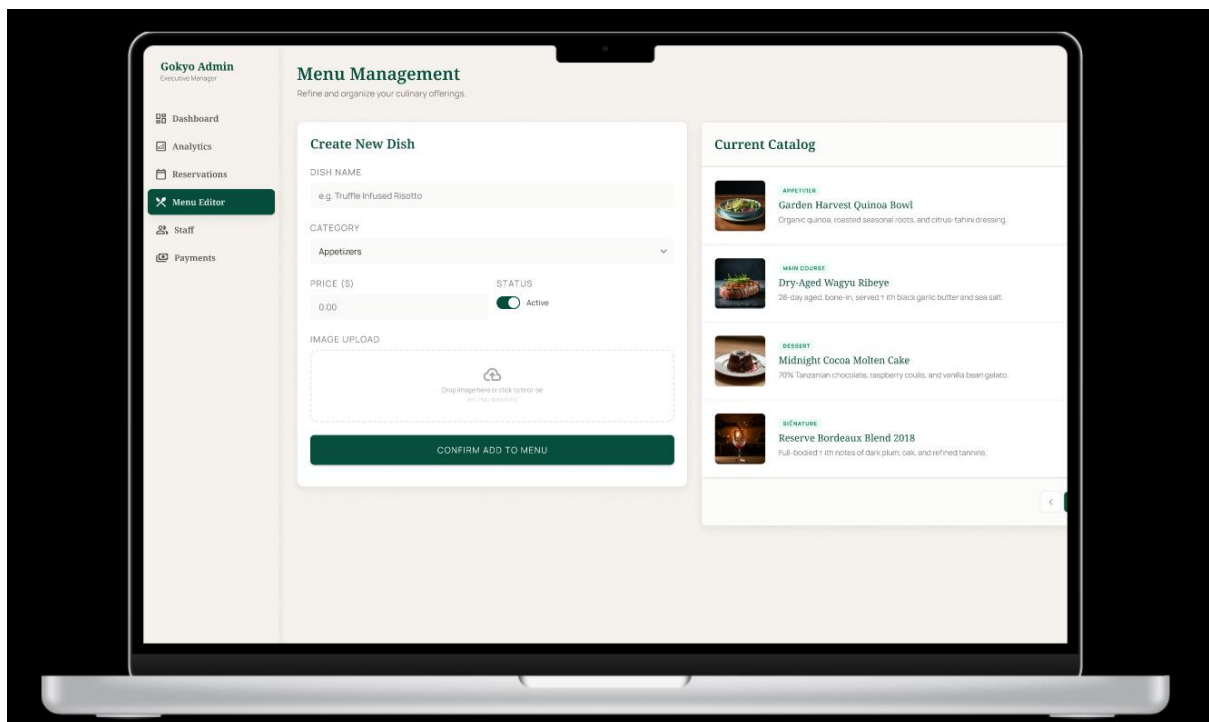
Prototype 10: Order Confirmation.

After a successful payment, the order confirmation page shows a success banner, order reference number, itemised order summary, and estimated preparation time. In case of home delivery orders, the live order status tracker displays the status changes through the stages, which include Order Confirmed, Being Prepared, Out for Delivery and Delivered.



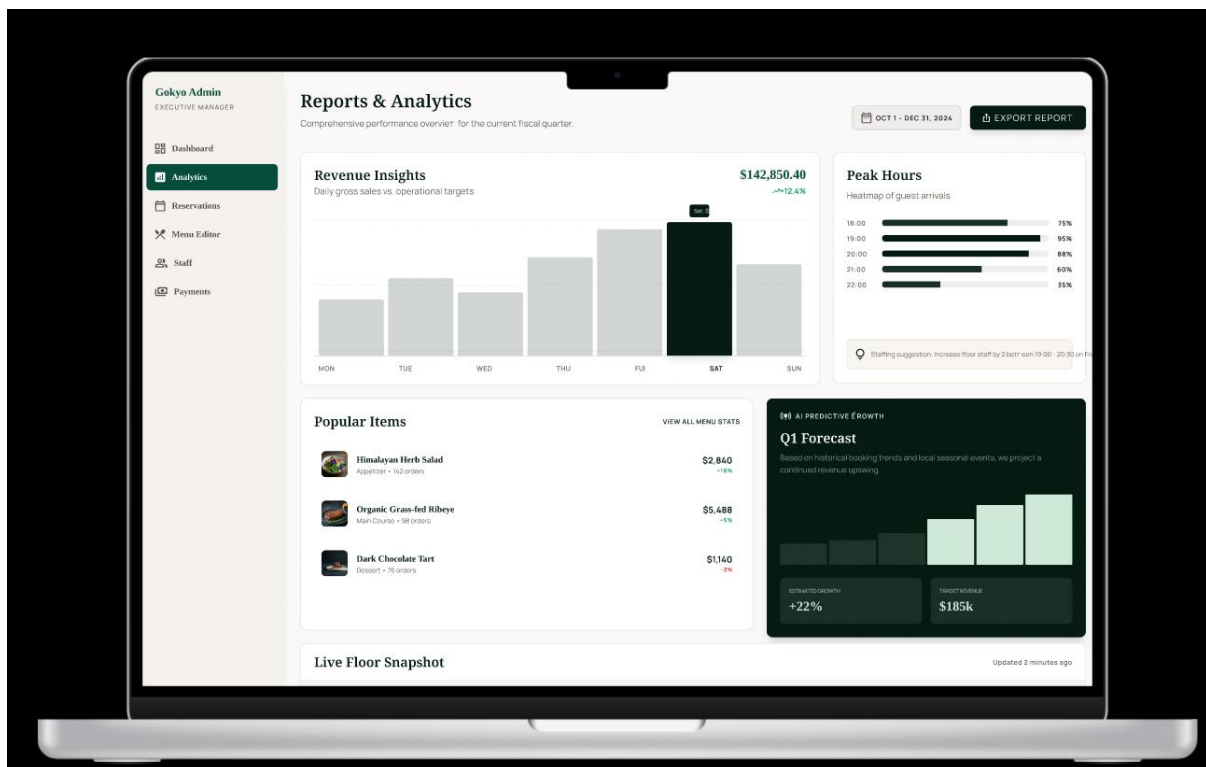
Prototype 11: Menu Management Page (Admin)

The management page of the admin menu has a searchable and filterable table of all menu items, which has columns of item name, category, price and availability switch. There is an Add New Item button that opens an inline form or modal to add entries. The current row items have Edit and Delete action buttons. The bulk availability updates may be done through a category-level switch to do quick menu updates in peak times.



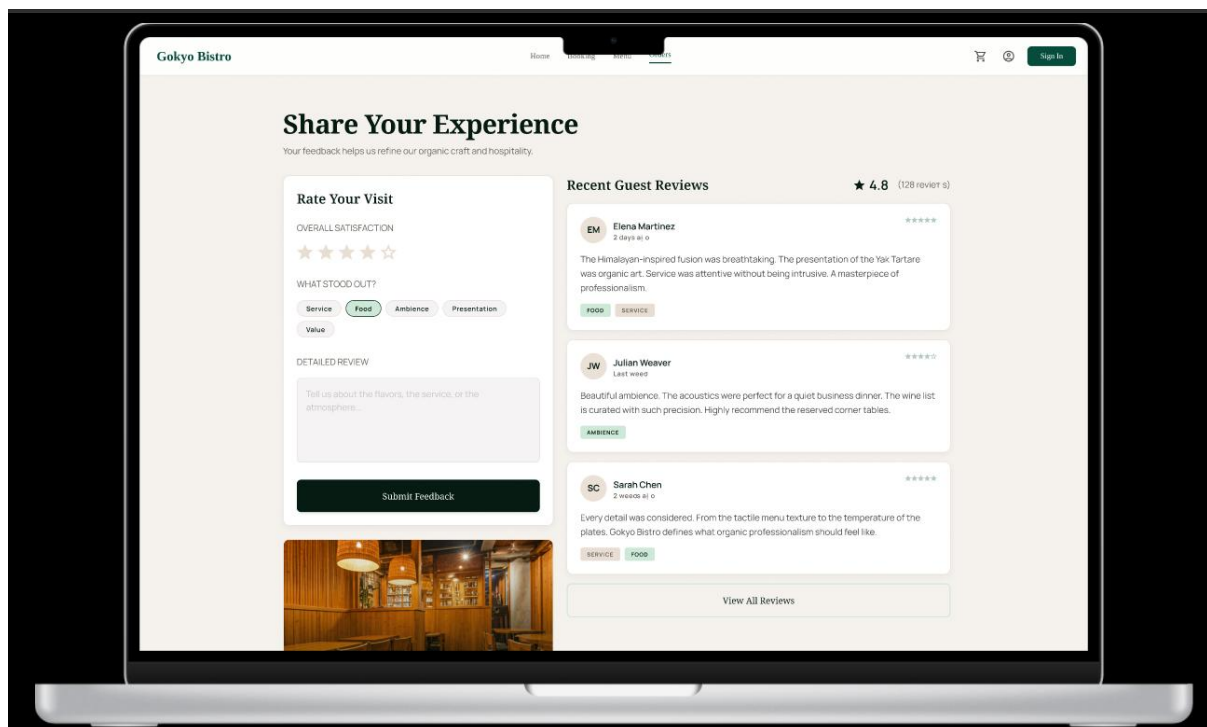
12 Prototypes: Financial Report Page (Admin)

The financial reporting page enables the Admin to choose a reporting period using a date range picker. Summary cards show the total revenue, transactions, average value of transaction and most popular menu items. A line chart will show the revenue trends within the period chosen. Another Prediction tab displays AI-estimated revenue forecasts in the upcoming quarters and confidence intervals, as well as suggested menu promotions.



Prototype 13: Rate Experience Page.

The feedback page will be available following a dining or delivery experience. Independent scores on Food Quality, Service, and Ambience are taken using three star-rating widgets. Qualitative feedback is possible through an optional, free-text comment box (up to 500 characters). Ratings made are instantly recorded in the feedback history of the user. Submission is completed by a Submit Feedback button and a thank-you overlay indicates that the comment has been received.



Prototype 14: Page of offers and discounts (Admin).

In the offers management page, there is a list of current, planned, and expired offers with status badges. A button of Add New Offer opens the creation form that asks to enter the offer title, description, type of discount (percentage or fixed), types of menu categories or specific items that the offer is relevant to, a start and end date, and the customer group that this offer is aimed at (all, member, premium member). The offers published are instantly displayed on the landing page, which is customer-facing.

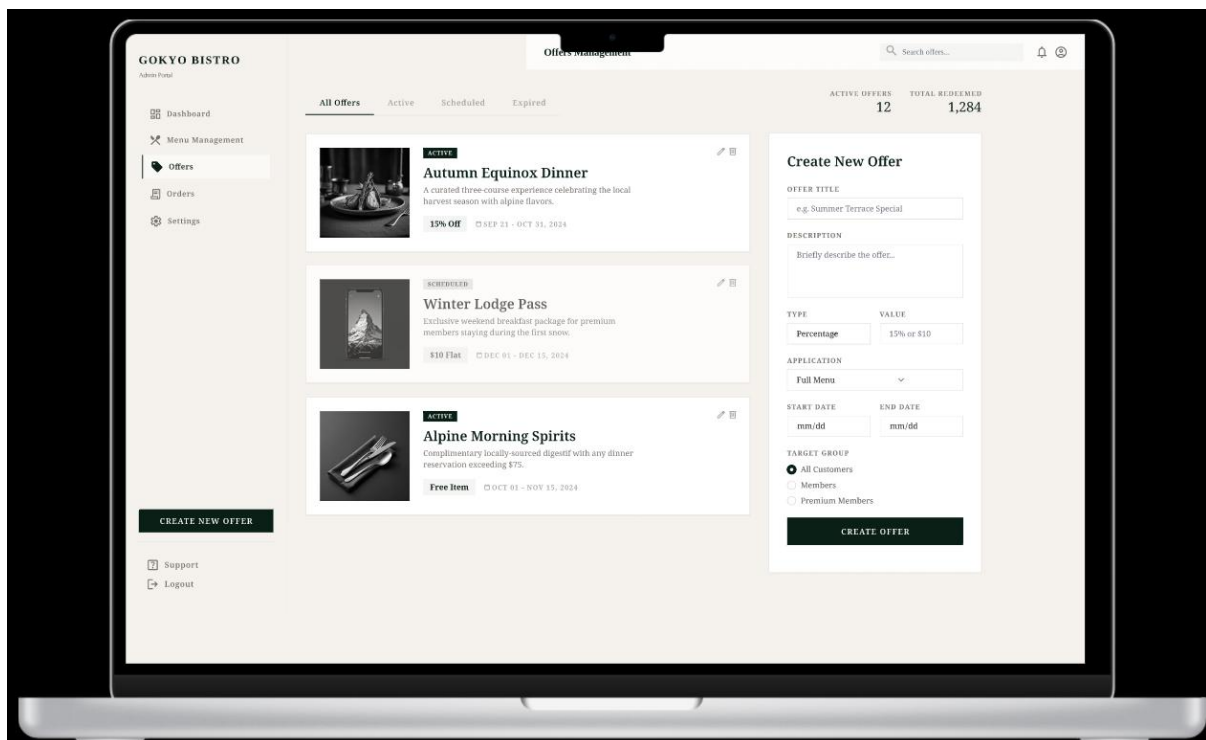


Figure 14: Prototype 14 Page of offers and discounts (Admin)

Prototype 15: Bill / Receipt Page.

A print-formatted receipt is shown on the bill page including the restaurant name and address, order/booking reference, date/time, list of charges charged under the item with quantity and price per unit, subtotal, tax paid (where appropriate), discounts and the total amount to pay. In the case of online orders, an electronic receipt is sent at the same time to the registered address of the member by email. This page allows the administration to initiate billing regarding dine in tables.

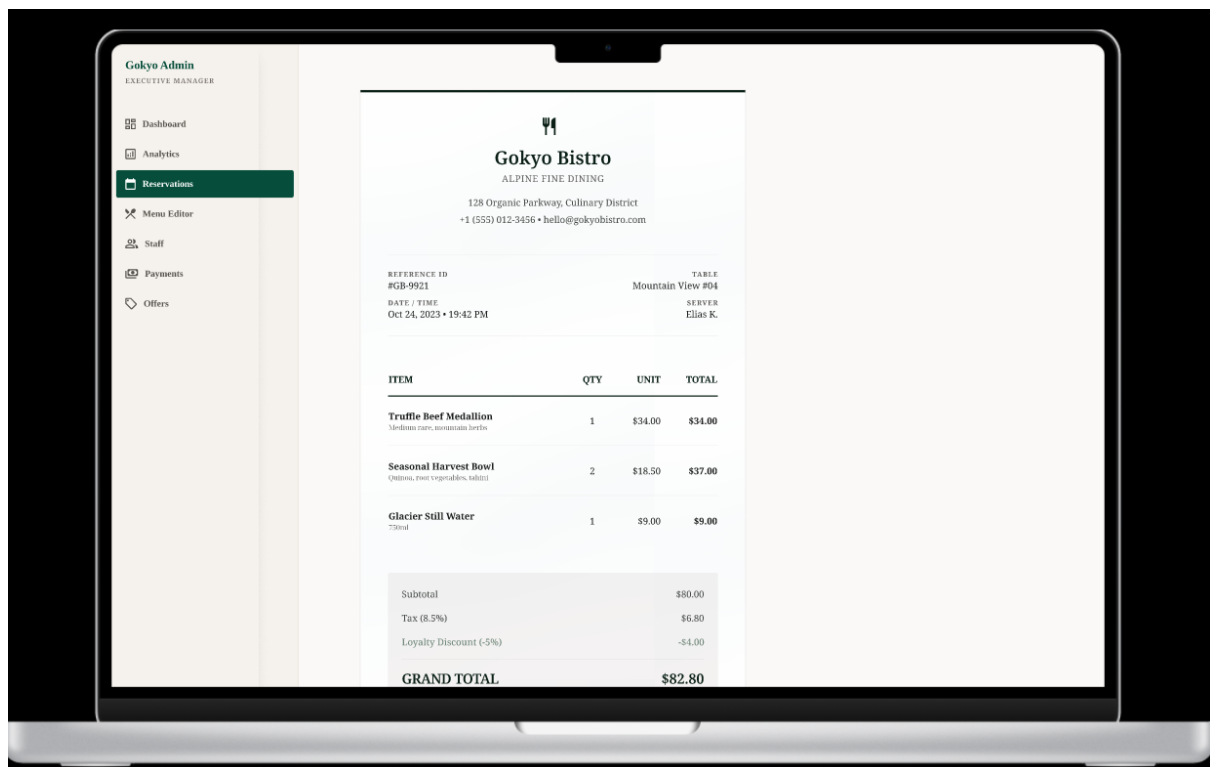


Figure 15: Prototype 15 Bill / Receipt Page

9 CONCLUSION

In this report, an Object-Oriented Analysis and Design artefact set of the proposed Gokyo Bistro Management System have been presented. The document, starting with a detailed Work Breakdown Structure that subdivided the project scope into eight sprint-sized work packages, and a Gantt Chart to schedule all activities over a one-year Agile-Scrum timeline, provided a strict project management framework on which the system development effort would be based.

The Use Case Model identified the functional requirements of the Gokyo Bistro system in ten major use cases which were backed by high level descriptions of all use cases and detailed descriptions of Book Table and Place Order. These artefacts determine the extent of actor-system interactions and are the requirements baseline on which design and implementation will be based. The Book Table use case Collaboration Diagram demonstrated the object-level interaction structure of the booking workflow and defined five entity objects and their relationships passing messages. The Place Order use case Activity Diagram was used to model the control flow of the order process on two swimlanes, including branch activities, parallel activities, and sequential activities.

The Analysis Class Diagram, a synthesis of the domain knowledge of all the use cases, was created as an 18-class structural model with inherited, composed, aggregated, and associated relationships with multiplicities described. This diagram gives the outline of the implementation of class hierarchy and database schema. The Further Development Plan provided a clear roadmap of design artefacts to working software based on all the decisions made with the help of the Agile-Scrum methodology and a specific technological stack of React/Node.js/PostgreSQL, MVC design pattern, four-level testing strategy, and systematic post-deployment maintenance programmed.

Lastly, fifteen user interface prototypes depicted the desired appearance and feel of the key features of the system and gave a concrete foundation of stakeholder feedback and system design refinement. Collectively these artefacts indicate that a carefully planned, OOAD-based solution offers a solid baseline to the provision of a stable, scalable, and maintainable management solution to the operational problems of Gokyo Bistro and its further development.

10 References

Anon. (20176) *Unified Modeling Language (UML) Specification* [Online].

Cockburn, A. (2001) *Writing Effective Use Cases*. Addison-Wesley.

Erich Gamma, R.H.J.V. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*.

IBM. (2020) [Online].

Kerzner, H. (2017) *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. Wiley.

Sommerville, I. (2016) *Software Engineering*. Pearson.

Sutherland, K.S.&.J. (2020) *The Scrum Guide*.

WorkBreakdownStructure.com. (2019) *Work Breakdown Structure* [Online]. Available from: <https://www.workbreakdownstructure.com/> [Accessed 30 April 2026].